



**INSTITUTE FOR ADVANCED COMPUTING AND
SOFTWARE DEVELOPMENT, AKURDI, PUNE**

“STAYSPHERE – Smart Room Rental Platform”

PGCP-AC Feb 2026

Submitted By:

Group No: 47

Roll No.	Name of Student
262105	Uddhav Gawali
262033	Gaurav Jadhav

Mr. Jiva Farkade
Project Guide

Mr. Anil Sharma
Centre Coordinator

ABSTRACT

StaySphere is a smart room rental platform designed to simplify property discovery, booking and payment management for owners and tenants. The platform integrates a React.js front-end with a Java Spring Boot backend built on a microservices architecture, allowing property search, booking and payment concerns to be developed, scaled and maintained independently. Security is enforced through Spring Security with JWT-based authentication and role-based authorization for Admin, Owner and Tenant users, and all inter-service communication.

StaySphere is built to streamline the process of listing, discovering and booking rental rooms. Tenants can search and filter properties, review rent, deposit, occupancy type and facilities, and submit booking requests that owners can approve or reject. Once a booking is approved, payment is handled by a dedicated Payment microservice developed using Java and Spring Boot, which keeps transaction processing isolated from the core rental application. The project addresses the practical challenges faced by tenants and property owners in coordinating discovery, booking and payment through a single, integrated platform.

ACKNOWLEDGEMENT

I take this occasion to thank God, almighty for blessing us with his grace and taking our endeavor to a successful culmination. I extend my heartfelt thanks to our esteemed guide, **Mr. Jiva Farkade** for providing me with the right guidance and advice at the crucial juncture and showing me the right way. I sincerely thank our respected Centre Co-Ordinator, **Mr. Anil Sharma**, for allowing us to use the available facilities. I would also like to thank the other faculty members at this occasion. Last but not least, I would like to thank my friends and family for the support and encouragement they have given me during our work.

Uddhav Gawali - 260241220200

Gaurav Jadhav - 260241220068

Table of Contents

Sr.No	Description	Page No.
1	Introduction	1
1.1	Purpose	3
1.2	Scope	3
1.3	Objective of StaySphere	3
1.4	Functionalities Provided by StaySphere	4
2	Software Requirement Specification	6
2.1	Functional Requirements	6
2.2	Non-Functional Requirements	8
3	Diagrams	9
3.1	ER Diagram	9
3.2	Use Case Diagram	10
3.3	Data Flow Diagram	11
3.4	Activity Diagram	12
3.5	Class Diagram	15
3.6	Sequence Diagram	16
4	Database Design	17
4.1	Design	17
4.2	Tables	18
5	Snapshots	20
6	Conclusion	29
7	Future Scope	30
8	References	31

1. INTRODUCTION

StaySphere: A Smart Room Rental Platform. It is a platform designed to simplify property discovery, booking and payment management for tenants and property owners. The platform uses React.js for a responsive front-end, and Java with Spring Boot for a set of modular backend microservices. Security is enforced through Spring Security with JWT-based authentication and role-based authorization, and all client requests are routed through a Spring Cloud API Gateway.

StaySphere is tailored to meet the needs of tenants searching for accommodation and owners managing their properties, ensuring accurate property information, a clear booking workflow, and a straightforward payment process. This separation of concerns across microservices helps keep the booking and payment flows modular, fault-isolated and independently scalable.

In the StaySphere system, **Admin**, **Owner** and **Tenant** roles play crucial roles in the management and operation of the platform. Each role comes with specific responsibilities, permissions, and access levels to ensure that the system operates smoothly and securely.

1. Admin Role

The admin role oversees the overall functioning of the StaySphere platform. Admins are responsible for monitoring users, properties and bookings across the system to ensure the platform runs smoothly and securely.

❖ Responsibilities:

- User Management:
 - o Manage tenant and owner accounts.
 - o Monitor account status – ACTIVE, SUSPENDED or DEACTIVATED.
- Property & Booking Oversight:
 - o Monitor properties listed by owners and their current status.
 - o Monitor booking requests and their progress across the platform.
- Administrative Operations:
 - o Access administrative dashboards and platform-wide activity.
 - o Take administrative action on users or listings where required.

❖ Access Level:

- Full access to administrative features exposed by the platform.
- Visibility into user, property and booking data needed for platform oversight.

2. Owner Role

Owners are property holders who list their rooms or properties on StaySphere and manage bookings for them. Their access is scoped to the properties and bookings that belong to them.

❖ Responsibilities:

- Property Management:
 - o Add, update and delete property listings.
 - o Add property facilities and upload or manage property images.
- Booking Management:
 - o View incoming booking requests for their properties.
 - o Approve or reject booking requests raised by tenants.

❖ Access Level:

- Access limited to the properties and bookings owned by the logged-in owner.
- No access to platform-wide administrative functionality.

3. Tenant Role

Tenants are end users searching for rental accommodation. Their access is focused on discovering properties, submitting booking requests and completing payments.

❖ Responsibilities:

- Property Discovery:
 - o Search and filter properties by rent, occupancy type and facilities.
 - o View detailed property information before booking.
- Booking & Payment:
 - o Submit booking requests for a selected property.
 - o Track the status of submitted booking requests.
 - o Complete payment once a booking request is approved.

❖ Access Level:

- Access limited to their own bookings, payments and profile information.

Role-Based Access Control (RBAC)

The system implements Role-Based Access Control (RBAC) using Spring Security together with JWT-based authentication. This ensures that users only have access to the functionalities they need based on their role. For instance:

- Admins have visibility into platform-wide user, property and booking data.
- Owners can manage only the properties and bookings associated with their own account.
- Tenants can search properties and manage only their own bookings and payments.

Security Considerations

All three roles are secured through authentication and authorization mechanisms provided by Spring Security. This includes:

- **Authentication:** Verifying the identity of a user at login and issuing a JWT to represent the authenticated session.
- **Authorization:** Ensuring that a request is permitted for the role associated with the JWT before it reaches a downstream microservice.
- **Stateless Session Handling:** Using JWTs so that session state does not need to be maintained on the server between requests.

StaySphere's role-based structure ensures that responsibilities are clearly delegated between Admin, Owner and Tenant, and that each user interacts only with the parts of the system relevant to their role, without compromising the security of the platform.

1.1 Purpose

The purpose of the StaySphere system is to provide a smart and organized platform for renting rooms, connecting tenants who are searching for accommodation with owners who wish to list their properties. The system is designed to simplify property discovery, booking coordination and payment handling, reducing the manual effort typically involved in room rental.

1.2 Scope

The **scope of the StaySphere project** covers the design, development and deployment of a microservices-based room rental platform that integrates property listing, booking management and payment processing. The scope is defined by the functionalities offered to Admins, Owners and Tenants, the technologies used to build the platform, and the services that make up its backend.

1.3 Objective of StaySphere

The objectives of the StaySphere project outline the primary goals the system aims to achieve, guiding its design, implementation and operational focus.

1. Simplified Property Discovery
2. Streamlined Booking Workflow
3. Modular Payment Processing
4. Robust Security and Role-Based Access Control
5. Scalable Microservices Architecture
6. Centralized Request Routing via API Gateway

7. Efficient Data Retrieval and Filtering
8. Reliable Booking and Payment Status Tracking
9. Support for Platform Growth and Expansion

1.4 Functionalities Provided by StaySphere

1. User Management

- **User Registration and Login:**
 - o Tenants and Owners can create accounts, log in and manage their profiles.
 - o Secure authentication and authorization using Spring Security and JWT.
- **Role-Based Access Control:**
 - o Distinct Admin, Owner and Tenant roles, each with specific permissions.
- **Profile Management:**
 - o Users can update their personal information and account details.

2. Property Management

- **Property Listing:**
 - o Owners can add, update and delete properties, including rent, deposit, occupancy type and status.
- **Facilities & Images:**
 - o Owners can add property facilities and upload or manage property images.
- **Property Search:**
 - o Tenants can search and filter properties by rent, occupancy type and facilities, with results sorted and paginated.

3. Booking Management

- **Booking Requests:**
 - o Tenants can submit booking requests for a property they are interested in.
- **Approval Workflow:**
 - o Owners can view booking requests for their properties and approve or reject them.
- **Status Tracking:**
 - o Tenants can track the status of their booking requests through to confirmation.

4. Payment Management

- **Payment Microservice:**
 - o A dedicated Payment microservice, built with Java and Spring Boot, handles payment initiation and processing after a booking is approved.
- **Transaction Tracking:**
 - o The service maintains transaction information and payment status – INITIATED, SUCCESS, FAILED or REFUNDED.
- **Booking Confirmation:**

- o A booking is confirmed once the associated payment is completed successfully.

5. Admin Operations

- **User Oversight:**
 - o Admins can manage user accounts and monitor account status.
- **Platform Monitoring:**
 - o Admins can monitor properties and bookings across the platform through an administrative dashboard.

6. Security and Compliance

- **Authentication & Authorization:**
 - o JWT-based authentication with role-based authorization enforced by Spring Security.
- **Input Validation:**
 - o Validation of incoming request data across services, with centralized, meaningful error responses.

7. Integration and API Design

- **REST APIs:**
 - o RESTful APIs expose functionality across the User, Property, Booking and Payment services.
- **API Gateway:**
 - o A Spring Cloud API Gateway provides a single entry point and centralized routing to the underlying microservices.
- **API Documentation:**
 - o Swagger / OpenAPI documentation describes the available endpoints for each service.

2. SOFTWARE REQUIREMENT SPECIFICATION

The functional requirements for StaySphere outline the specific features and capabilities that the system must provide to meet the needs of Admins, Owners and Tenants. These requirements guide the development of each microservice and ensure that the final platform aligns with the intended booking and payment workflow.

2.1 Functional Requirements for StaySphere

1. User Management

- **User Registration:** The system shall allow tenants and owners to create an account by providing personal details such as name, email and password.
- **User Authentication:** The system shall authenticate users during login using their registered email and password, and issue a JWT on success.
- **Role-Based Access Control:** The system shall support role-based access, where Admin, Owner and Tenant users have different permissions.
- **Profile Management:** Users shall be able to view and update their profile information.

2. Property Management

- **Property Listing:** The system shall allow owners to add, update and delete properties, storing details such as rent, deposit, occupancy type, facilities and status.
- **Property Images:** The system shall allow owners to upload and manage one or more images per property.
- **Property Search:** The system shall allow tenants to search and filter properties by rent range, occupancy type and facilities, with results paginated and sorted.

3. Booking Management

- **Booking Request:** Tenants shall be able to submit a booking request for a property.
- **Booking Approval:** Owners shall be able to view booking requests for their properties and approve or reject them.
- **Booking Status Tracking:** Tenants shall be able to track the status of their booking requests.

4. Payment Management

- **Payment Initiation:** The system shall allow a tenant to initiate payment once a booking request has been approved.
- **Payment Processing:** The Payment microservice shall process the payment request and record the resulting transaction.
- **Payment Status:** The system shall track payment status as INITIATED, SUCCESS, FAILED or REFUNDED, and communicate the outcome back to the core application.
- **Booking Confirmation:** The system shall confirm a booking only after the associated payment status is recorded as SUCCESS.

5. Admin Operations

- **User Oversight:** The system shall allow admins to manage user accounts and monitor account status.
- **Platform Monitoring:** The system shall allow admins to monitor properties and bookings across the platform.

6. Security

- **Authentication and Authorization:** The system shall enforce JWT-based authentication and ensure that users can only access functionality permitted by their role.
- **Input Validation:** The system shall validate incoming request data and return meaningful error responses for invalid input.

2.2 Non-Functional Requirements for StaySphere

1. Performance

- **Response Time:**

The system shall respond to user actions within an acceptable time under normal operating conditions.

- **Efficient Data Retrieval:**

The system shall use indexed and paginated queries so that property search remains responsive as the number of listings grows.

2. Reliability

- **Fault Isolation:**

The Payment microservice shall be isolated from the core rental application so that a failure in payment processing does not bring down property browsing or booking.

- **Error Handling:**

The system shall handle errors gracefully across services and provide meaningful feedback through centralized exception handling.

3. Usability

- **User Interface:**

The system shall provide a clear, responsive interface for property search, booking and payment, built using React.js.

4. Maintainability

- **Modularity:**

The system shall be organized as independent microservices – User, Property, Booking and Payment – so that individual services can be updated without affecting the rest of the platform.

- **Layered Architecture:**

Each service shall follow a layered structure separating controllers, services, repositories and DTOs to keep the codebase maintainable.

5. Security

- **Data Protection:**

The system shall protect user credentials and payment-related data through Spring Security and standard hashing practices.

Hardware and Software Interfaces

Back-end Server Configuration:

- Java 21
- Spring Boot 3.5
- Spring Web
- Spring Data JPA / Hibernate
- Spring Security
- JWT (JWT Authentication)
- MySQL 8.0
- Springdoc OpenAPI / Swagger
- REST APIs

Front-end Client Configuration:

- React 18
- Vite
- React Router DOM
- Axios
- HTML5, CSS3, JavaScript

3. DIAGRAMS

3.1 Entity Relationship Diagram:

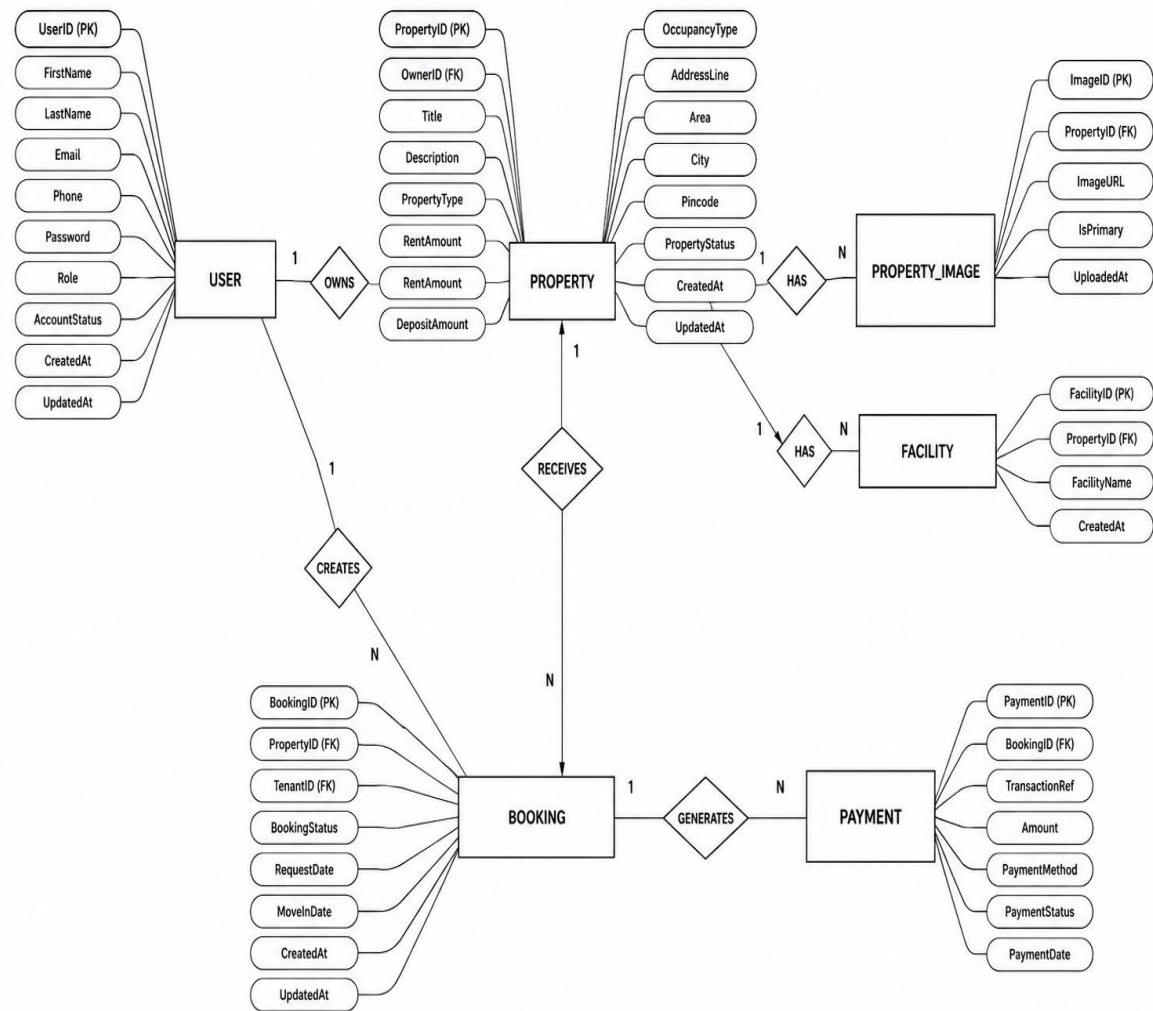


Fig. ER Diagram for StaySphere

3.2 Use Case Diagram:

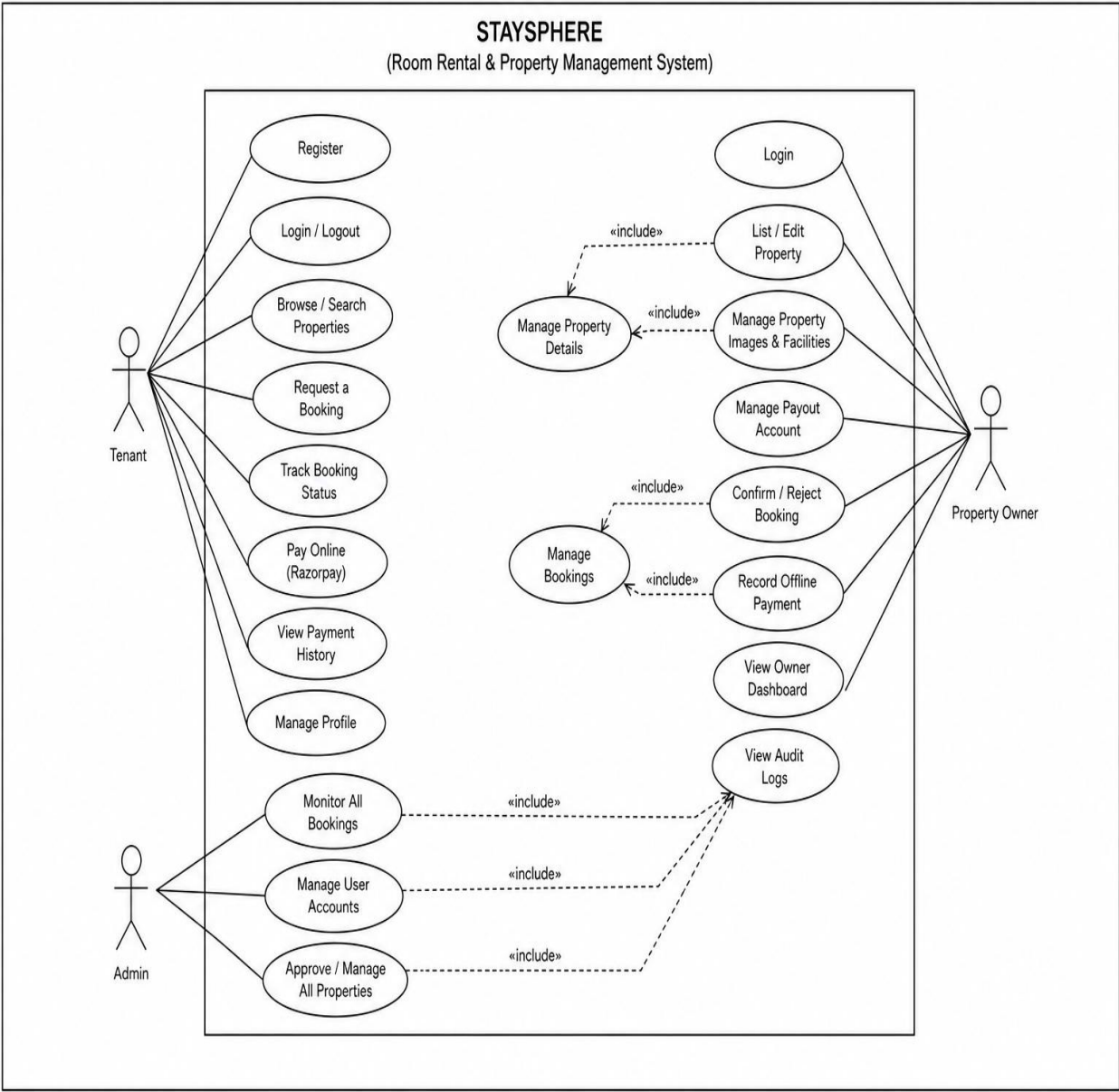


Fig. Use Case Diagram for StaySphere

3.3 Data Flow Diagram:

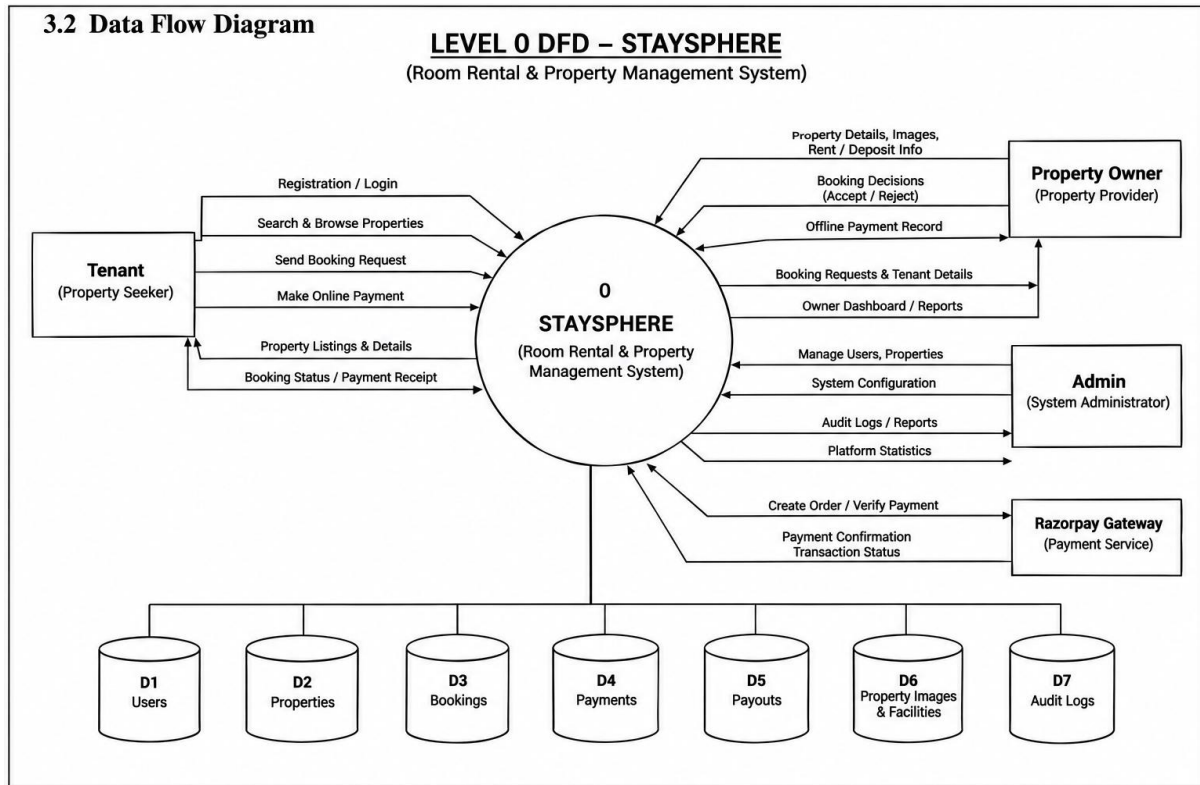


Fig 3.3a: DFD Level 0 — Context Diagram

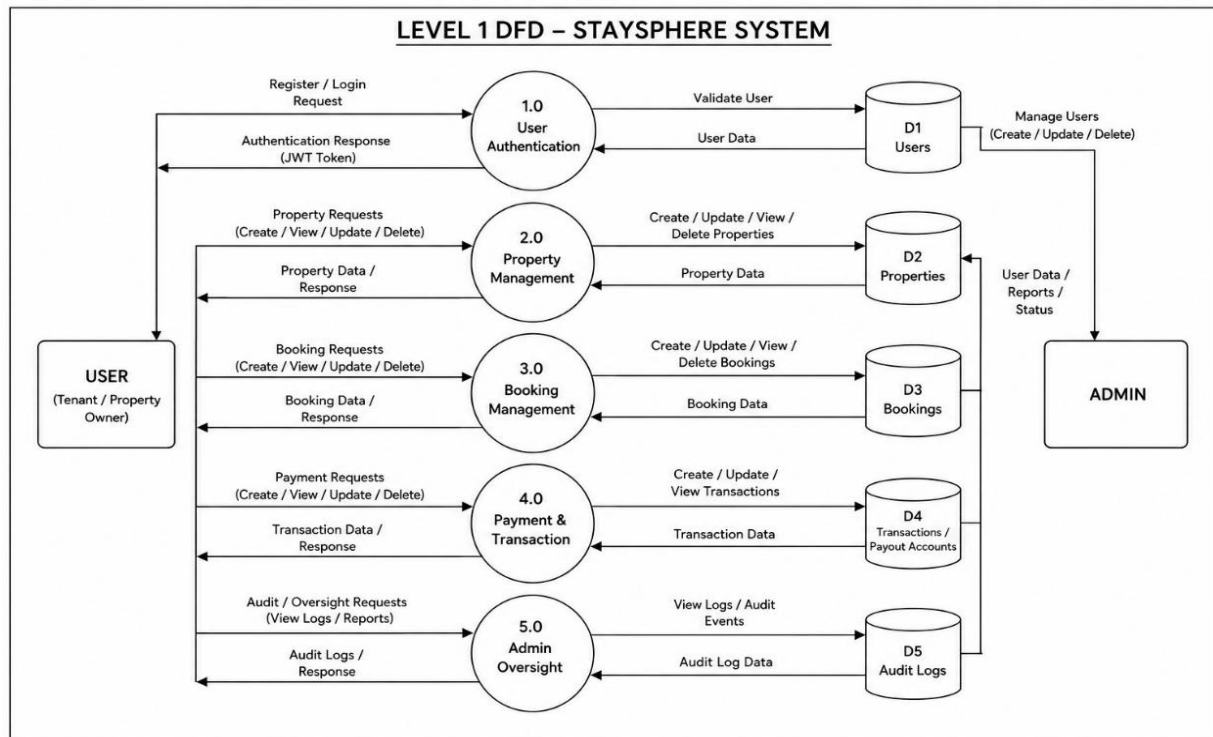


Fig 3.3b: DFD Level 1

3.4 Activity Diagram:

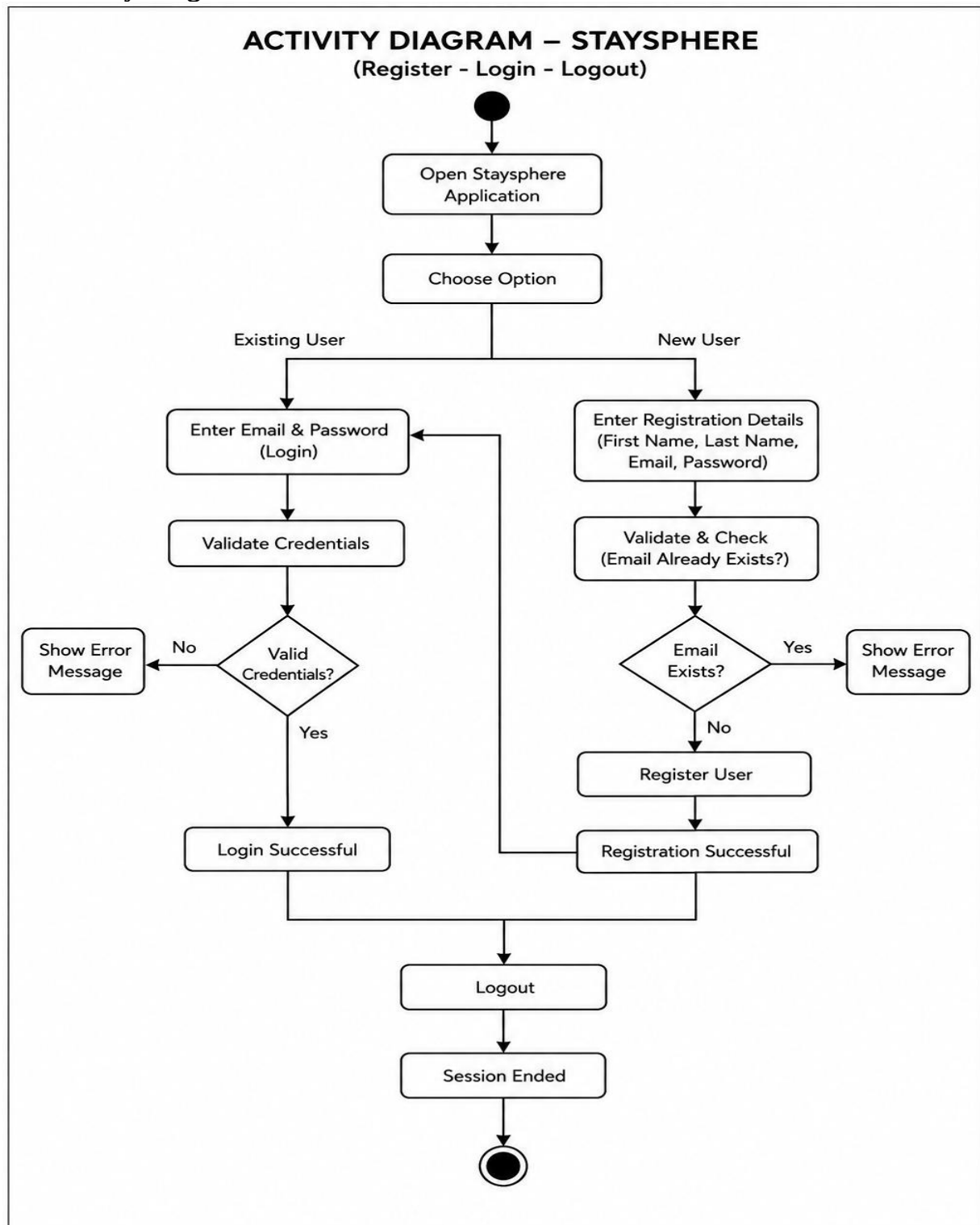
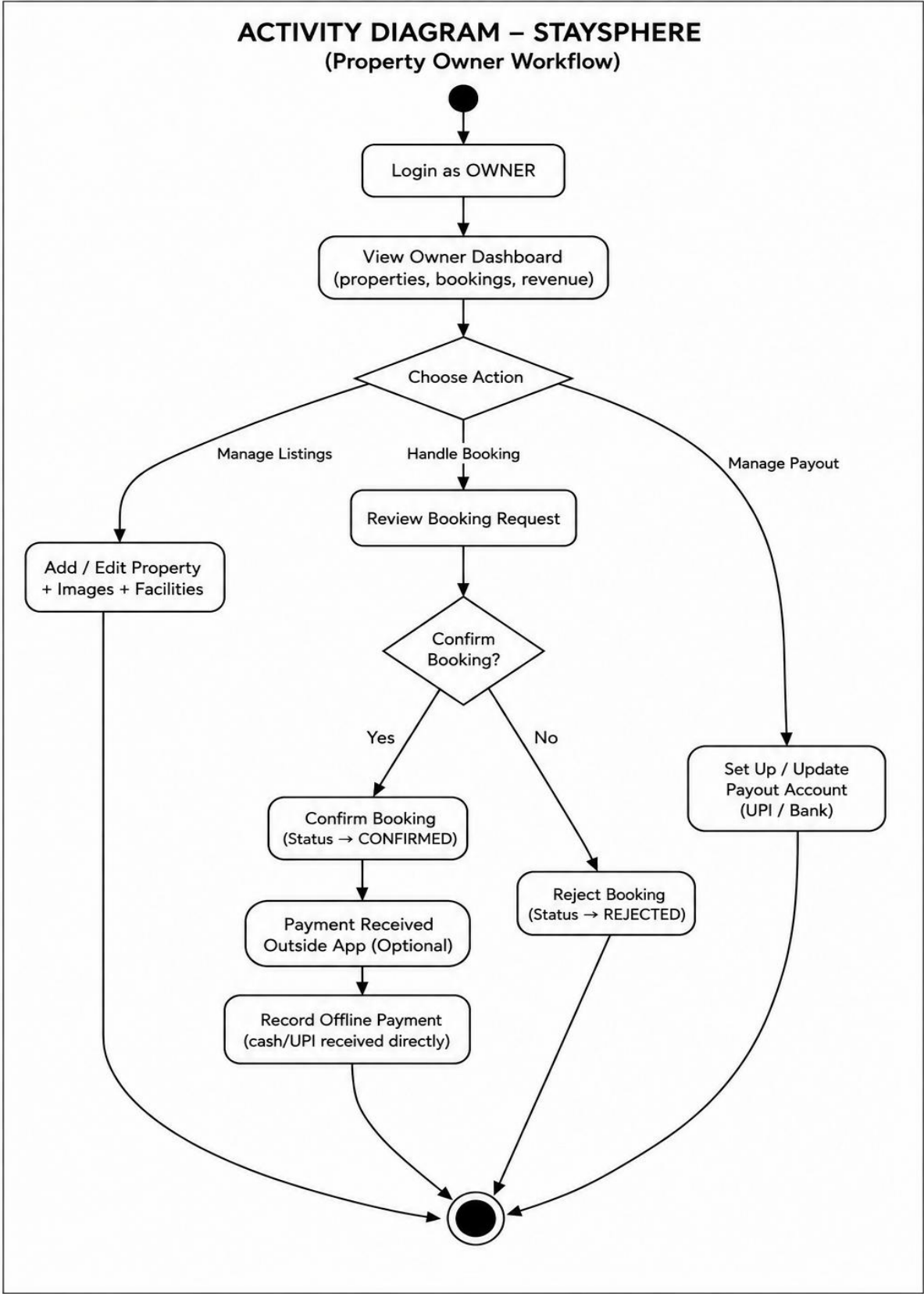


Fig 3.4a: Login Activity Diagram



Owner Activity Diagram

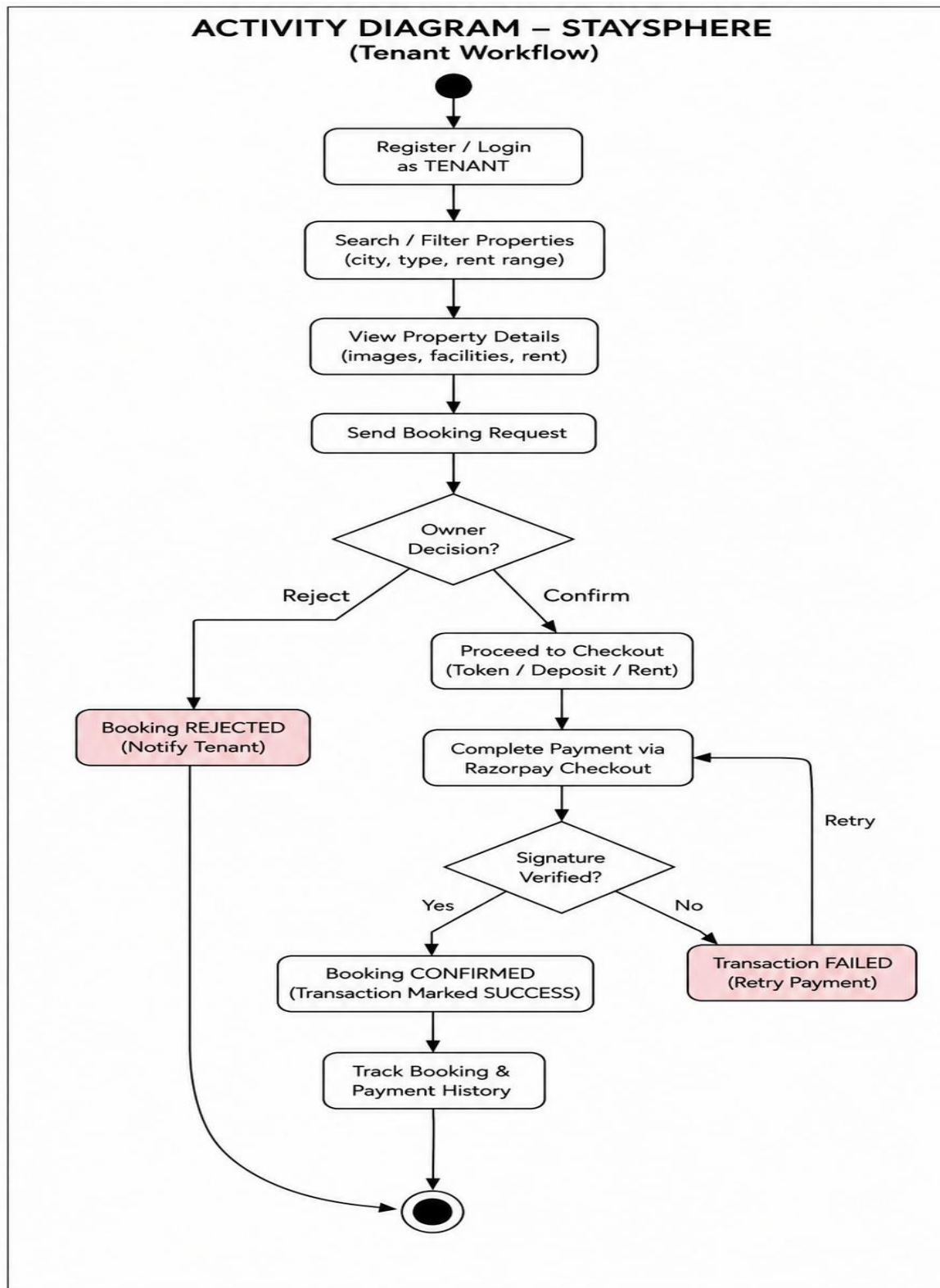


Fig 3.4c: Tenant Activity Diagram

3.5 Class Diagram:

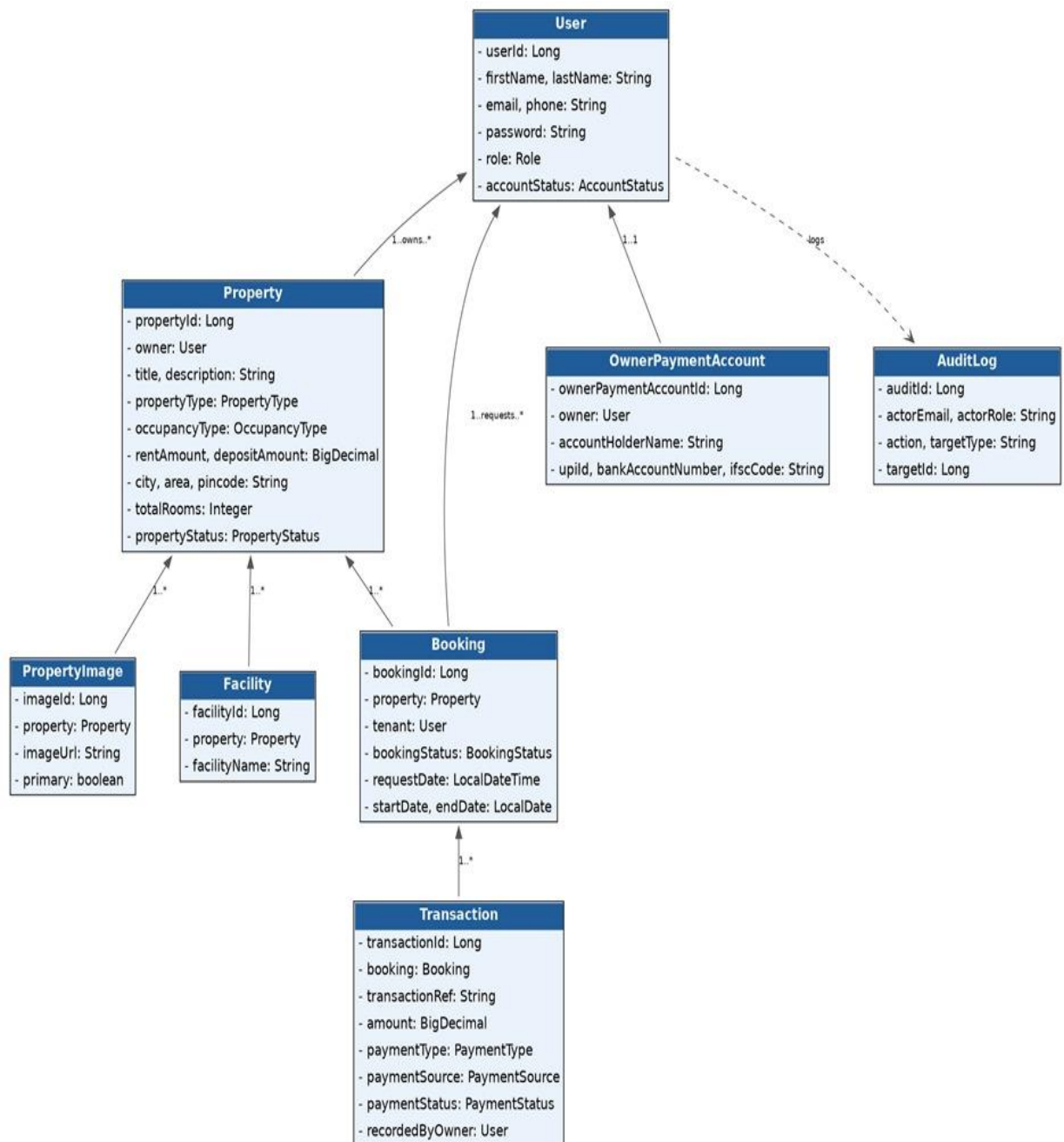


Fig. Class Diagram for StaySphere

3.6 Sequence Diagram:

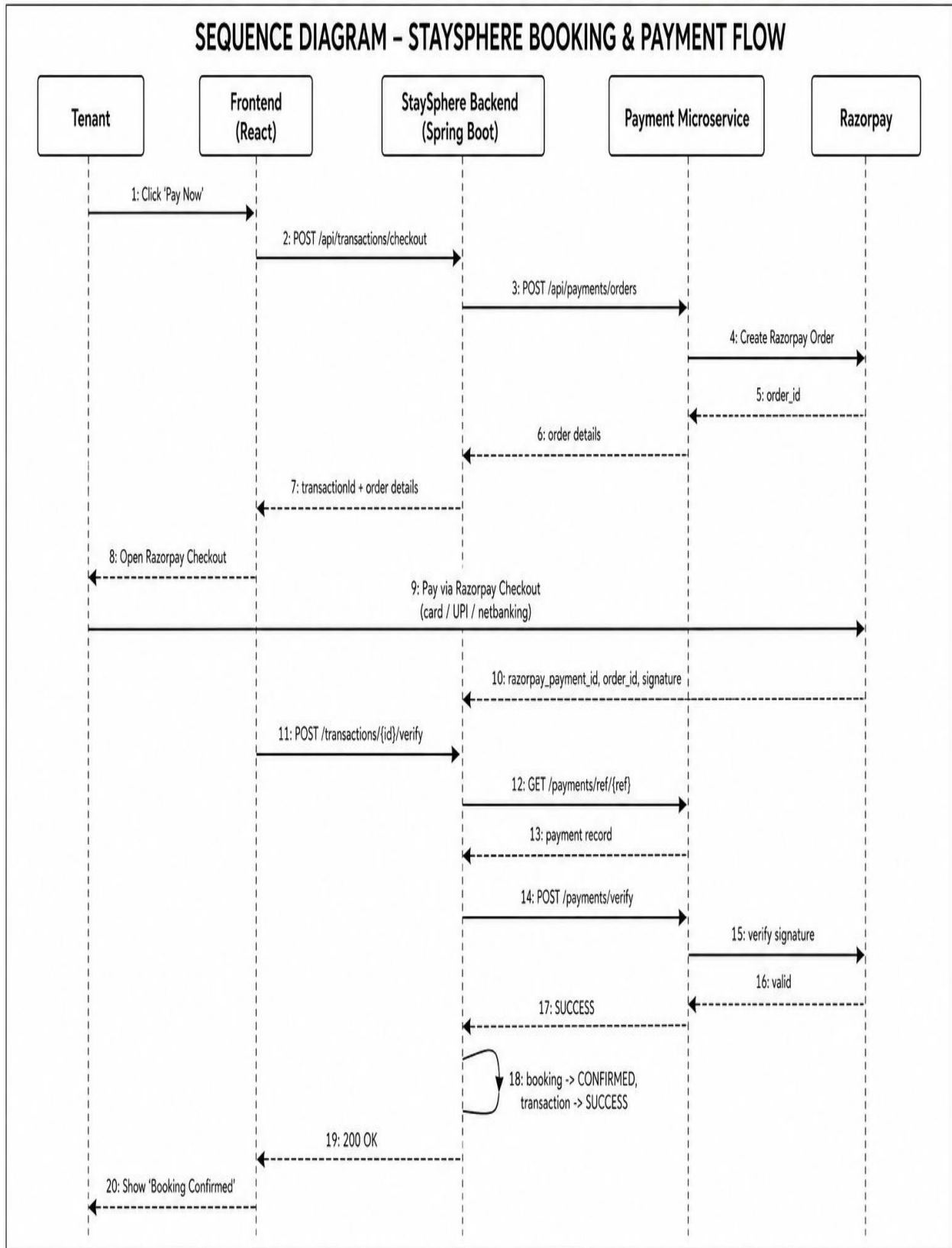


Fig. Sequence Diagram for StaySphere

4. DATABASE DESIGN

4.1 Design:

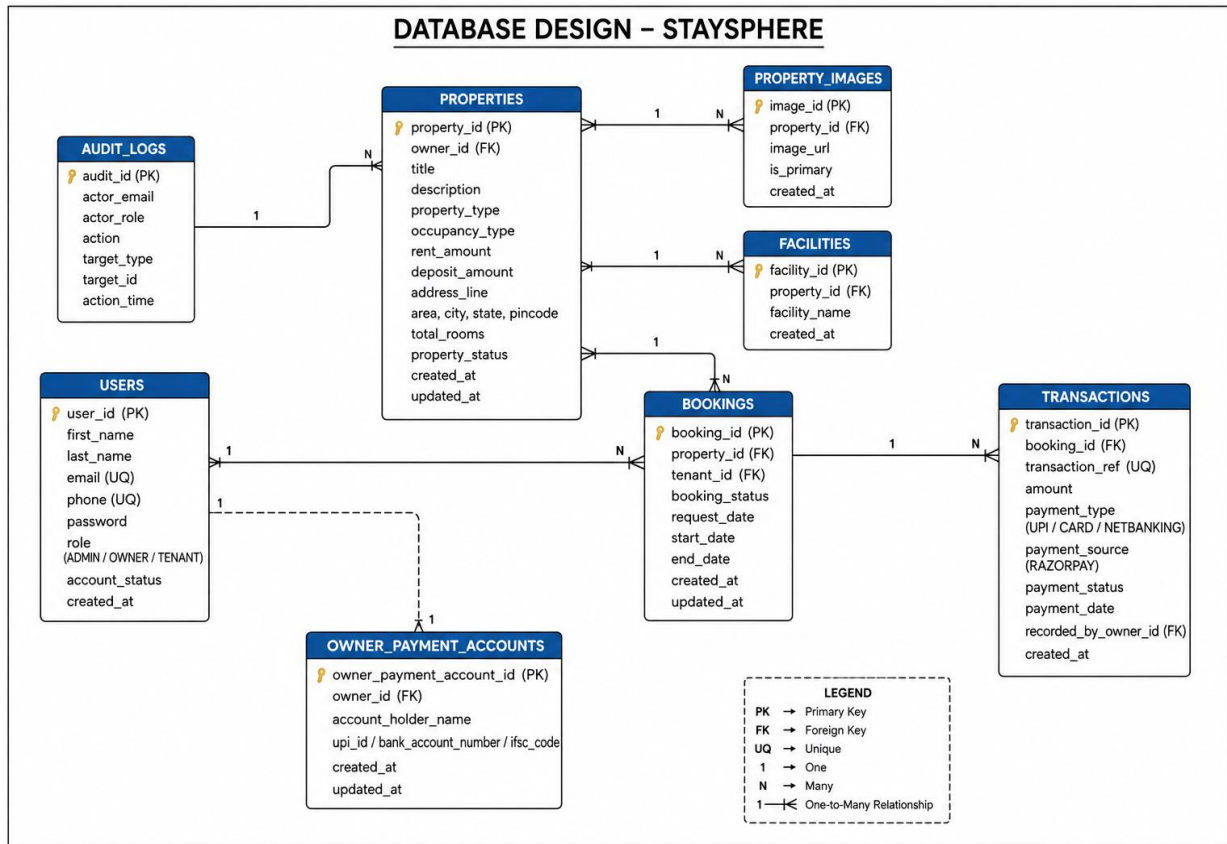


Fig. Database Design for StaySphere

4.2 Tables:

The following table structures depict the database design. Indexes were introduced on frequently queried attributes – such as property location, rent range and occupancy type – to reduce unnecessary table scans during property search operations.

Column	Type	Constraint / Notes
user_id	BIGINT (PK)	Auto-increment identity
first_name, last_name	VARCHAR(50)	Not null
email	VARCHAR(150)	Not null, unique
phone	VARCHAR(15)	Not null, unique
password	VARCHAR(255)	Not null, BCrypt hash
role	VARCHAR(20)	ADMIN / OWNER / TENANT
account_status	VARCHAR(20)	ACTIVE / SUSPENDED / DEACTIVATED
created_at, updated_at	DATETIME	Auditing timestamps

Table 1: users

Column	Type	Constraint / Notes
property_id	BIGINT (PK)	Auto-increment identity
owner_id	BIGINT (FK → users)	Not null; ON DELETE RESTRICT
title, description	VARCHAR(150) / TEXT	description nullable
property_type	VARCHAR(30)	ROOM / PG / FLAT / HOSTEL
occupancy_type	VARCHAR(20)	SINGLE / SHARED
rent_amount, deposit_amount	DECIMAL(10,2)	Not null
address_line, area, city, state, pincode	VARCHAR	Not null
total_rooms	INTEGER	Default 1; bookable-room count
property_status	VARCHAR(20)	ACTIVE / INACTIVE

Table 2: properties

Column	Type	Constraint / Notes
image_id	BIGINT (PK)	Auto-increment identity
property_id	BIGINT (FK → properties)	Not null; ON DELETE CASCADE
image_url	VARCHAR(500)	Not null
is_primary	BOOLEAN	Default false; one primary per property
uploaded_at	DATETIME	Set on insert

Table 3: property_images

Column	Type	Constraint / Notes
facility_id	BIGINT (PK)	Auto-increment identity
property_id	BIGINT (FK → properties)	Not null; ON DELETE CASCADE
facility_name	VARCHAR(100)	Free-text, e.g. Wi-Fi, Parking
created_at	DATETIME	Set on insert

Table 4: facilities

Column	Type	Constraint / Notes
booking_id	BIGINT (PK)	Auto-increment identity
property_id	BIGINT (FK → properties)	Not null; ON DELETE RESTRICT

tenant_id	BIGINT (FK → users)	Not null; ON DELETE RESTRICT
booking_status	VARCHAR(20)	REQUESTED / PAYMENT_PENDING / CONFIRMED / REJECTED / CANCELLED
request_date	DATETIME	Not null
start_date	DATE	Not null
end_date	DATE	Nullable — open-ended booking

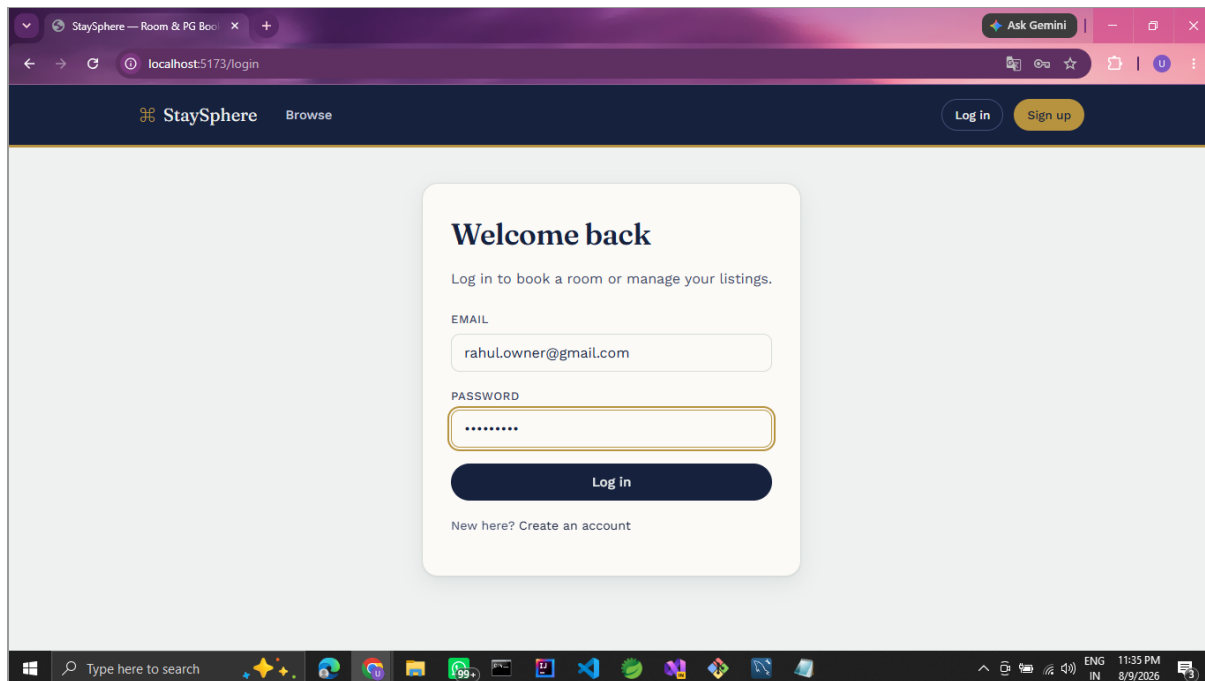
Table 5: bookings

Column	Type	Constraint / Notes
transaction_id	BIGINT (PK)	Auto-increment identity
booking_id	BIGINT (FK → bookings)	Not null; ON DELETE RESTRICT
transaction_ref	VARCHAR(100)	Not null, unique
amount	DECIMAL(10,2)	Not null
payment_type	VARCHAR(20)	TOKEN / DEPOSIT / RENT
payment_source	VARCHAR(20)	ONLINE (default) / OFFLINE
payment_status	VARCHAR(20)	PENDING / SUCCESS / FAILED
payment_date	DATETIME	Nullable until settled
recorded_by_owner_id	BIGINT (FK → users)	Set only for OFFLINE payments
notes	VARCHAR(255)	Optional, offline payments

Table 6: transactions

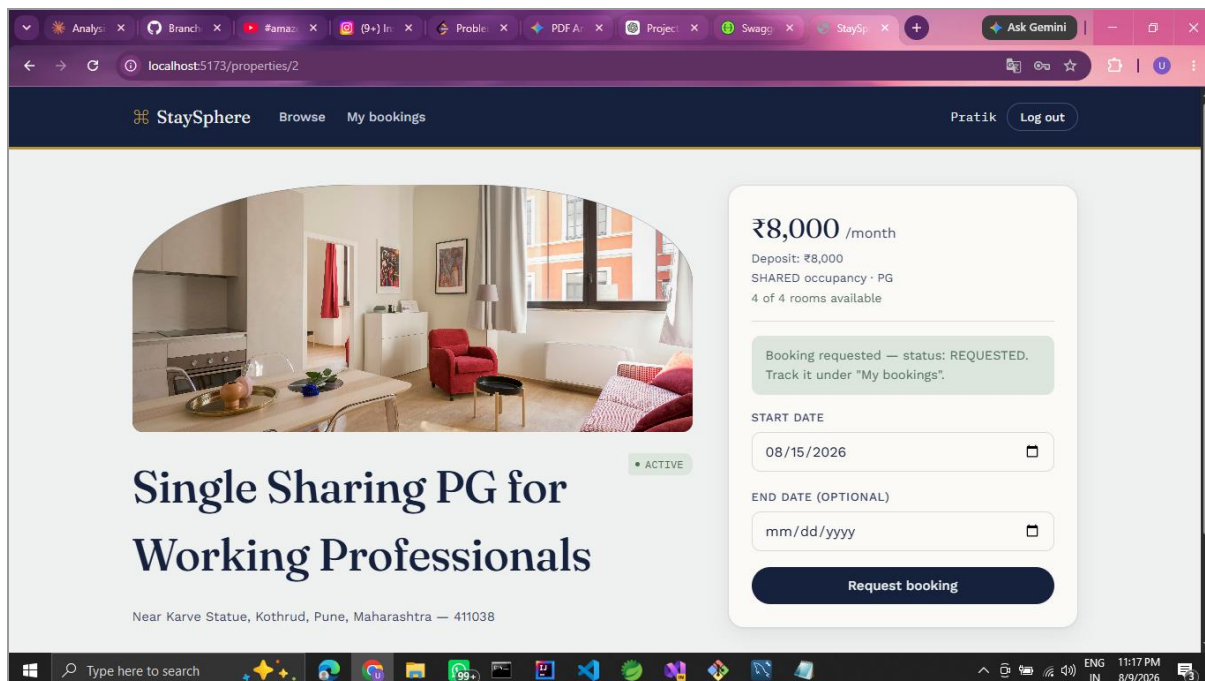
5. SNAPSHOTS

Login Page:



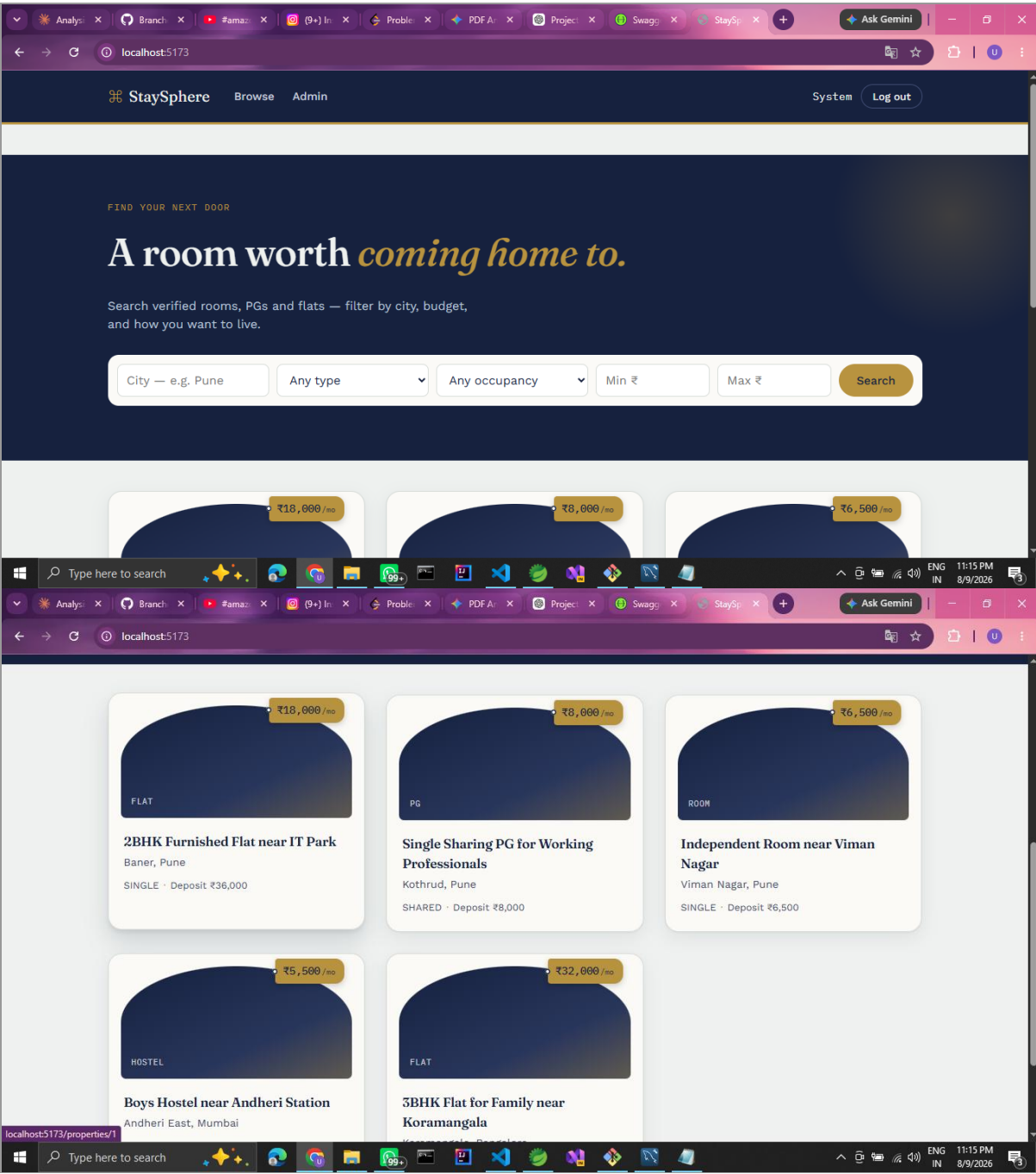
The screenshot shows a web browser window with the URL `localhost:5173/login`. The page features a dark blue header with the StaySphere logo, a "Browse" link, and "Log in" and "Sign up" buttons. The main content area has a light gray background with a white login card. The card displays "Welcome back" and "Log in to book a room or manage your listings." Below this are input fields for "EMAIL" (containing `rahuLowner@gmail.com`) and "PASSWORD" (masked with dots). A "Log in" button is at the bottom of the card, and a link "New here? Create an account" is below it. The browser's taskbar at the bottom shows various application icons and the system clock indicating 11:35 PM on 8/9/2026.

Tenant Dashboard:



The screenshot shows a web browser window with the URL `localhost:5173/properties/2`. The page features a dark blue header with the StaySphere logo, "Browse" and "My bookings" links, and a "Log out" button for user "Pratik". The main content area has a light gray background. On the left, there is a large image of a modern living room. Below the image, the text "Single Sharing PG for Working Professionals" is displayed, followed by "Near Karve Statue, Kothrud, Pune, Maharashtra — 411038". A green "ACTIVE" badge is next to the property title. On the right, a white card displays the rental details: "₹8,000 /month", "Deposit: ₹8,000", "SHARED occupancy · PG", and "4 of 4 rooms available". Below this, a green box states "Booking requested — status: REQUESTED. Track it under 'My bookings'." There are input fields for "START DATE" (containing "08/15/2026") and "END DATE (OPTIONAL)" (containing "mm/dd/yyyy"). A "Request booking" button is at the bottom of the card. The browser's taskbar at the bottom shows various application icons and the system clock indicating 11:17 PM on 8/9/2026.

Property Search:



Property Details:

Analytics Branch Amazon (9*) In Problem PDF A Project Swag StaySp Gemini

localhost5173/properties/1

StaySphere Browse Admin System Log out



ACTIVE

2BHK Furnished Flat
near IT Park

Plot 45, Baner Road, Baner, Pune, Maharashtra — 411045

₹18,000 /month

Deposit: ₹36,000
SINGLE occupancy · FLAT
2 of 2 rooms available

START DATE

mm/dd/yyyy

END DATE (OPTIONAL)

mm/dd/yyyy

Request booking

Type here to search

99+

ENG IN 11:15 PM 8/9/2026

Booking Request

Analytics Branch Amazon (9*) In Problem PDF A Project Swag StaySp Gemini

localhost5173/my-bookings

StaySphere Browse My bookings Pratik Log out

My bookings

Single Sharing PG for Working Professionals

Booking #1

Stay: 2026-08-15 (open-ended)

NOT PAID

REQUESTED

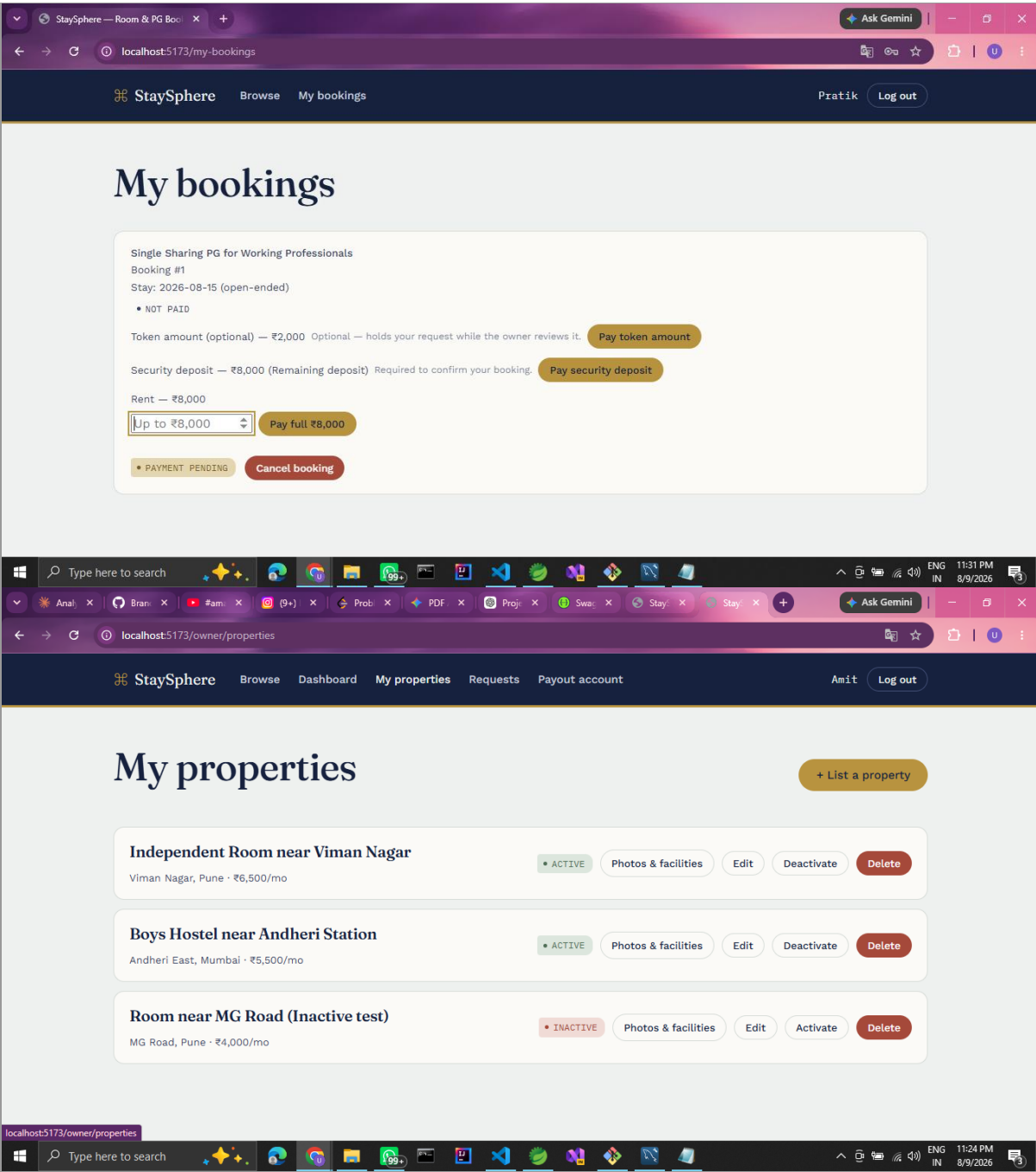
Cancel booking

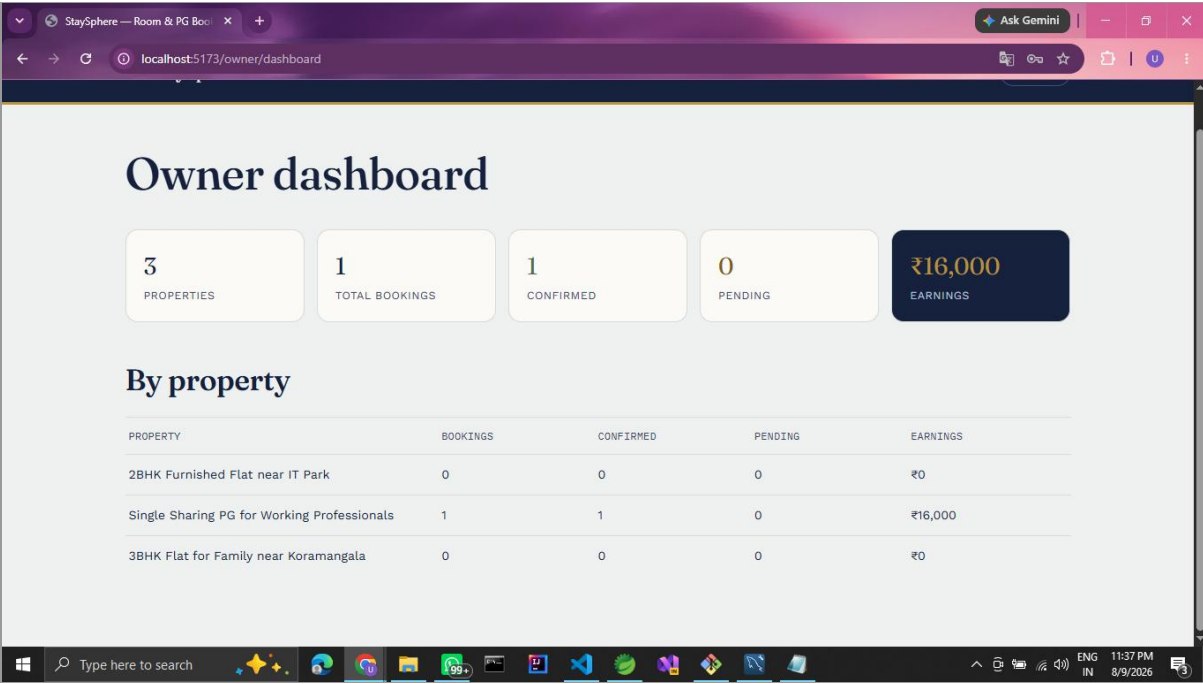
Type here to search

99+

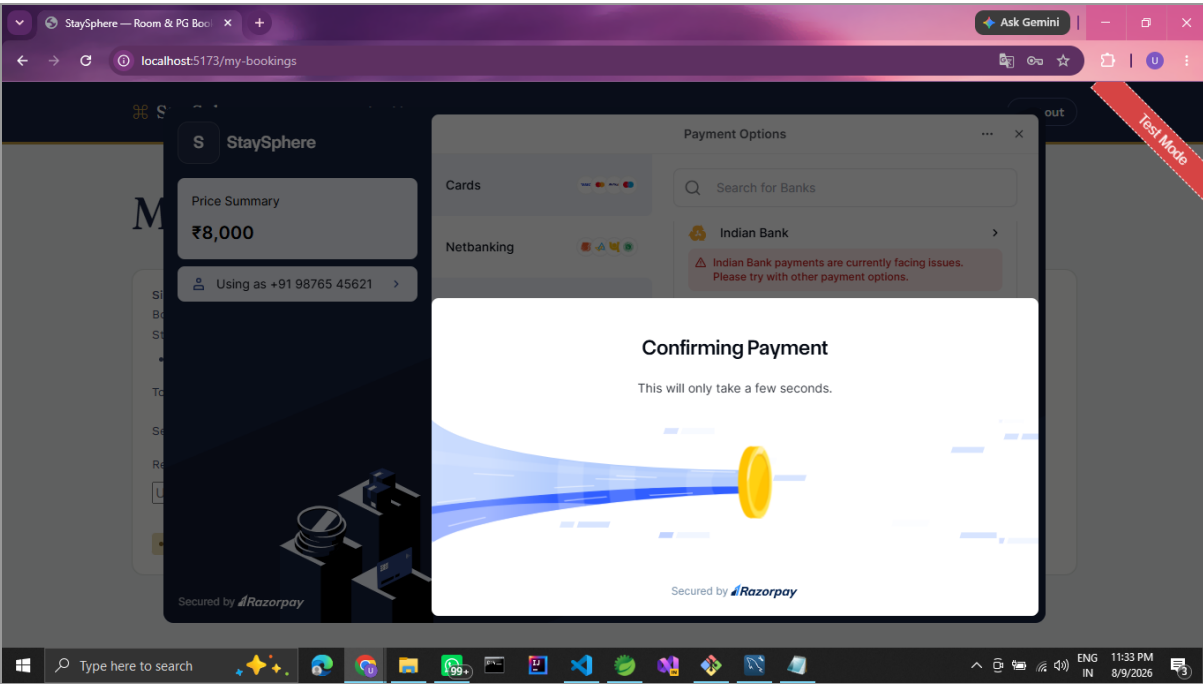
ENG IN 11:18 PM 8/9/2026

Owner Booking Management:

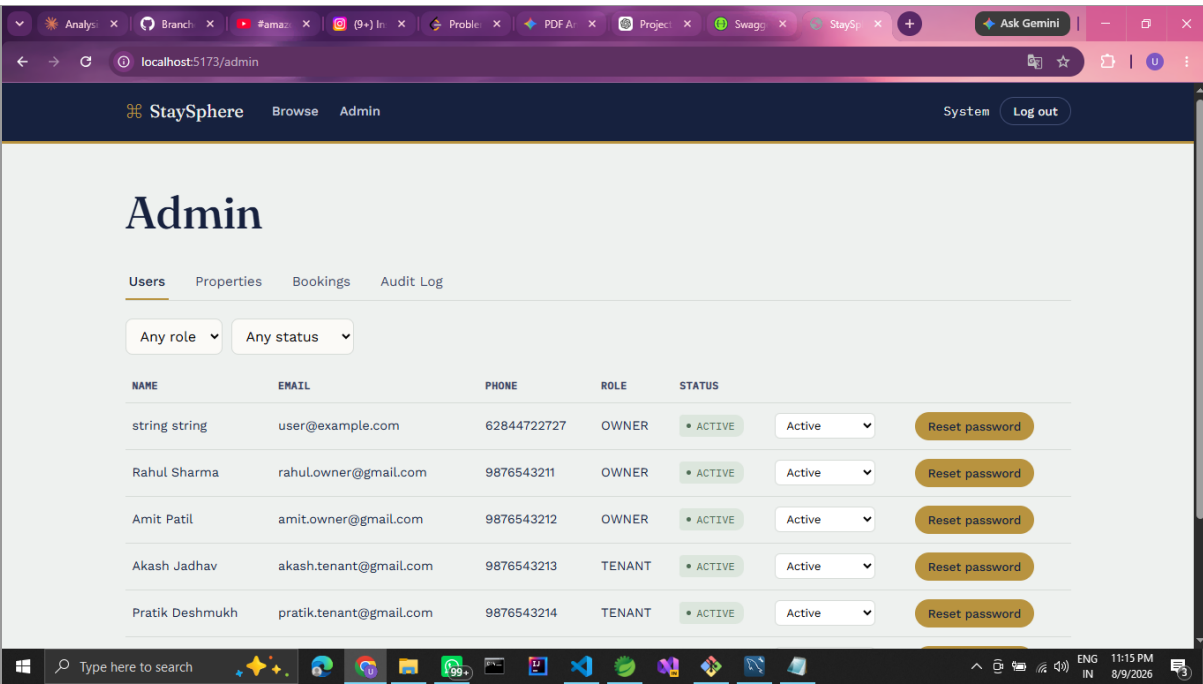


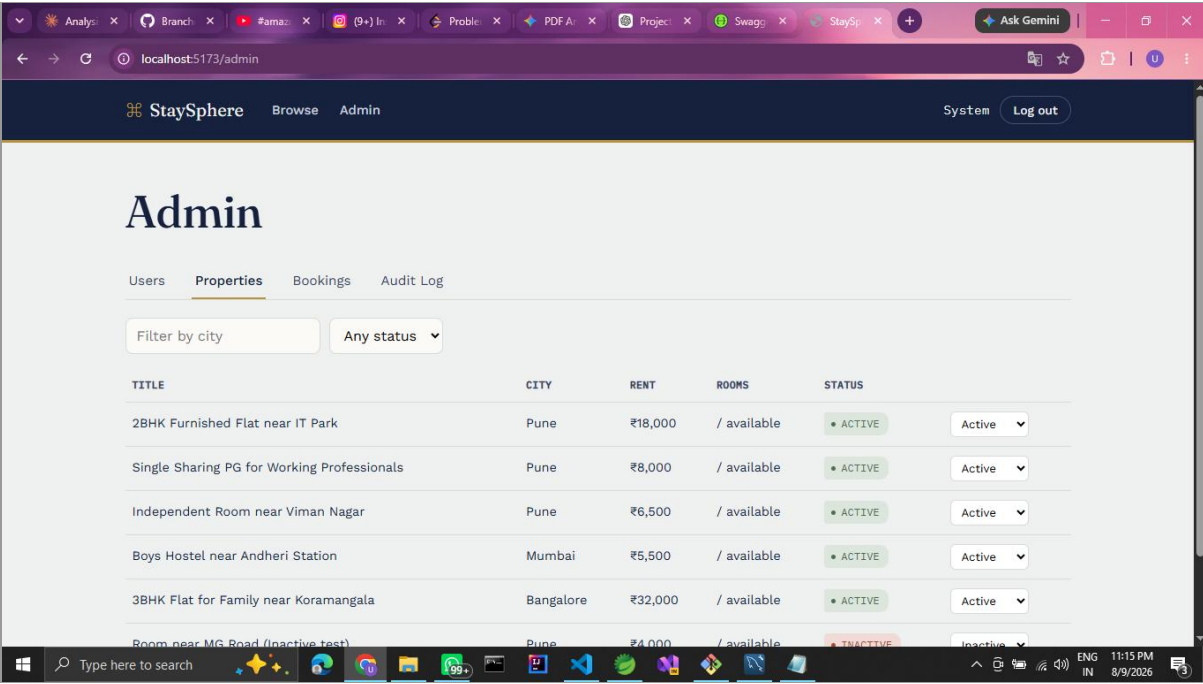


Payment Page:

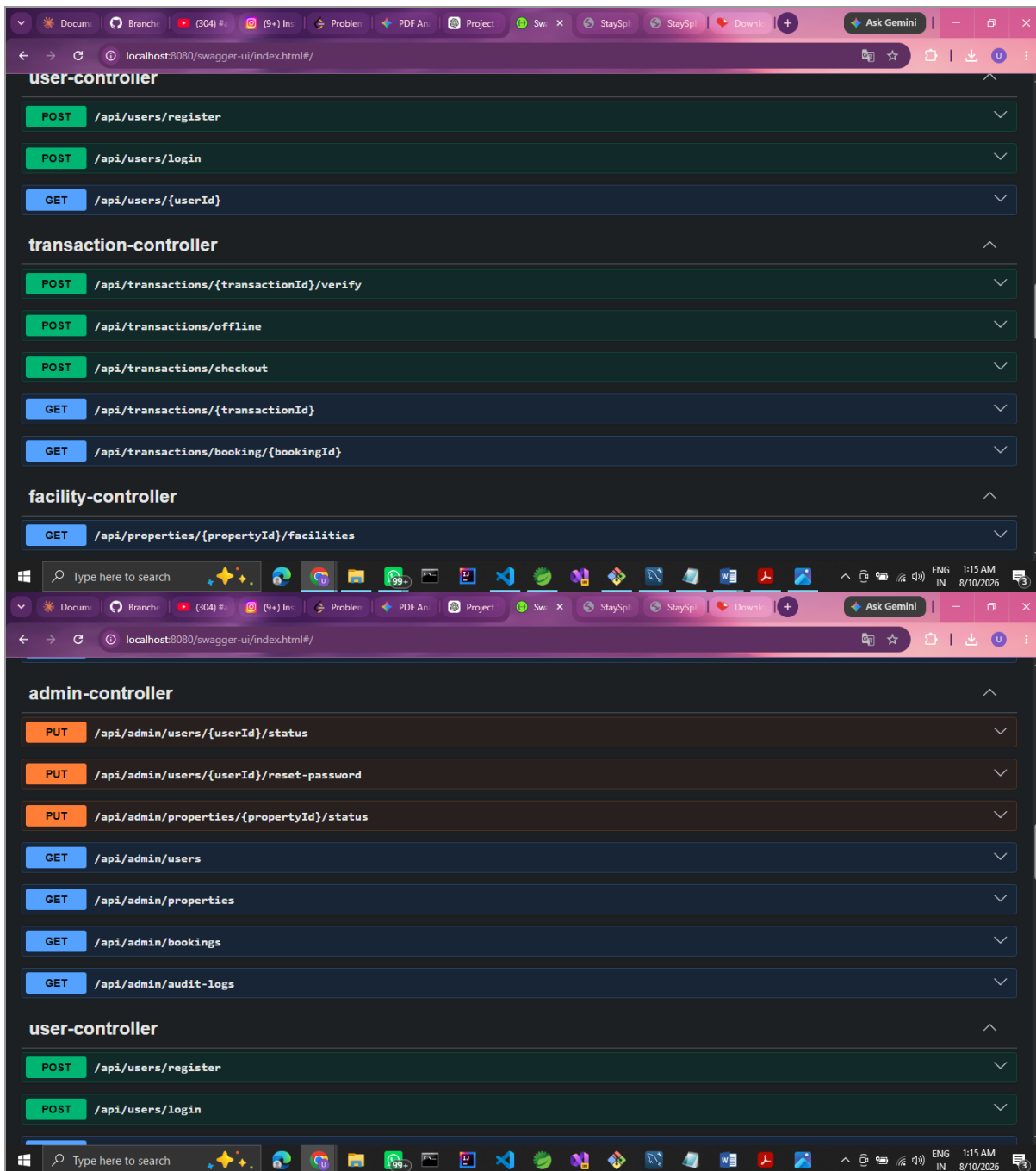


Admin Dashboard:





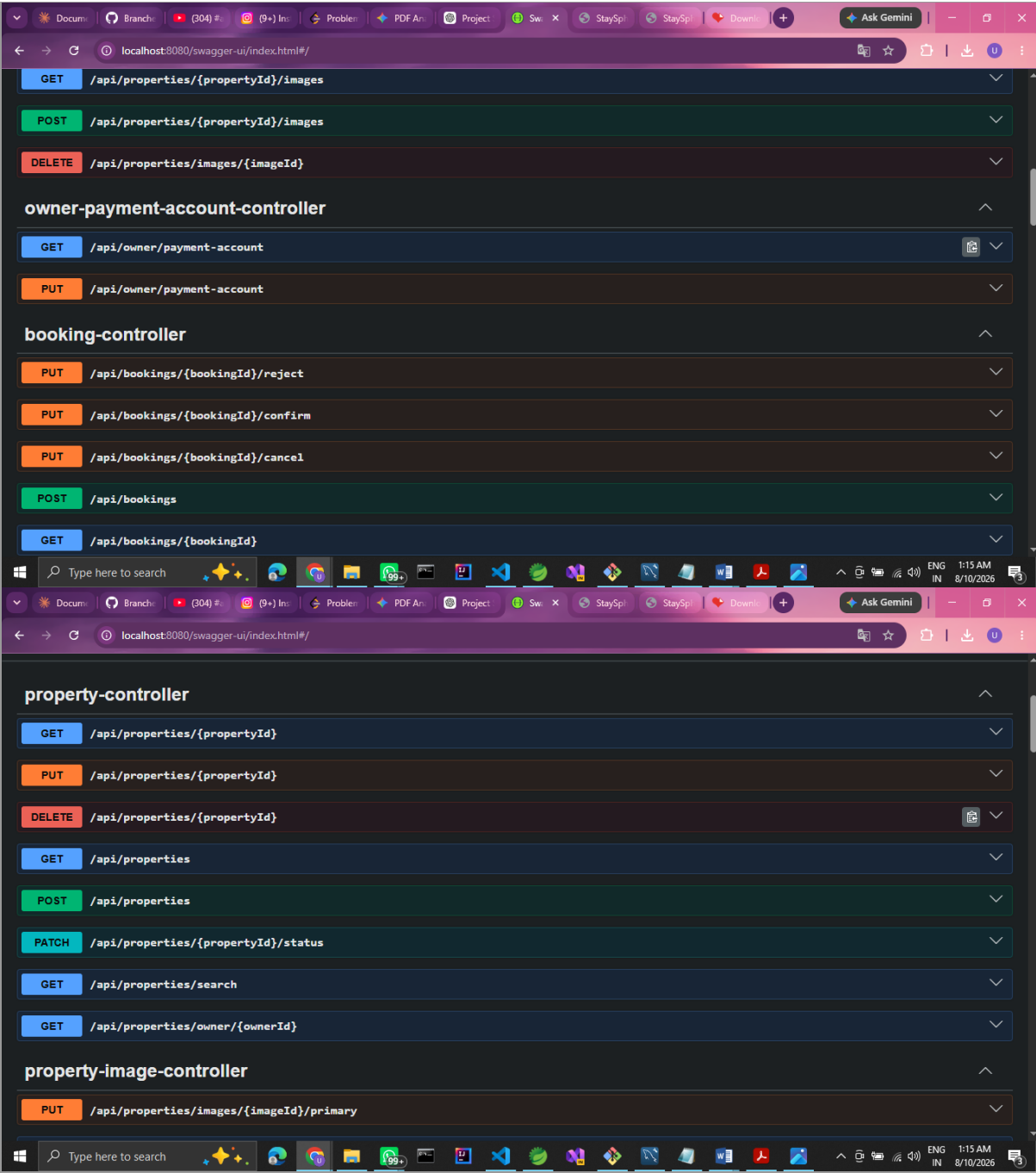
Swagger API Documentation:



The screenshot displays the Swagger UI for the StaySphere API, showing the following endpoints:

- user-controller**
 - POST `/api/users/register`
 - POST `/api/users/login`
 - GET `/api/users/{userId}`
- transaction-controller**
 - POST `/api/transactions/{transactionId}/verify`
 - POST `/api/transactions/offline`
 - POST `/api/transactions/checkout`
 - GET `/api/transactions/{transactionId}`
 - GET `/api/transactions/booking/{bookingId}`
- facility-controller**
 - GET `/api/properties/{propertyId}/facilities`
- admin-controller**
 - PUT `/api/admin/users/{userId}/status`
 - PUT `/api/admin/users/{userId}/reset-password`
 - PUT `/api/admin/properties/{propertyId}/status`
 - GET `/api/admin/users`
 - GET `/api/admin/properties`
 - GET `/api/admin/bookings`
 - GET `/api/admin/audit-logs`
- user-controller**
 - POST `/api/users/register`
 - POST `/api/users/login`

The interface includes a search bar at the top and a sidebar on the left for navigation. The bottom of the screen shows the Windows taskbar with various application icons and the system clock indicating 1:15 AM on 8/10/2026.



6. CONCLUSION

In conclusion, StaySphere addresses the practical requirements of a modern room rental workflow – property discovery, booking coordination and payment processing – through a modular, microservices-based platform. Its separation of concerns between the core rental application and the Payment microservice, along with role-based access control for Admins, Owners and Tenants, positions it well for continued development and extension.

The **StaySphere** project demonstrates a practical application of a microservices architecture using Java, Spring Boot and React.js, backed by Spring Security and JWT for authentication and authorization, and a Spring Cloud API Gateway for centralized request routing. The platform is designed to support tenants searching for accommodation and owners managing their listings, while keeping the booking and payment workflow clear and traceable through defined status transitions.

Its combination of a modular backend, role-based security and a dedicated payment service positions StaySphere as a solid foundation for further enhancement, such as richer search capabilities, notifications, or additional payment options, as the platform evolves.

7. Future Scope

Platform & User Experience

- In-app chat/messaging between tenant and owner

Booking & Property Management

- Virtual property tours (360° images / video walkthroughs)
- Review and rating system for properties and tenants

Payments & Finance

- Invoice/receipt generation for tax purposes

Notifications & Communication

- Push notifications for booking status, payment due dates, and offers
- Email/SMS integration for critical alerts

Scalability & Architecture

- Migrating to a full microservices architecture (already partially done with Payment)
- Caching layer (Redis) for faster property search
- Load balancing and horizontal scaling for high traffic

Security

- Document verification (Aadhaar/ID proof) for tenant and owner onboarding

7. REFERENCES

1. <https://docs.spring.io/spring-security/reference/>
2. <https://docs.spring.io/spring-boot/index.html>
3. <https://spring.io/projects/spring-cloud-gateway>
4. <https://react.dev/>
5. <https://dev.mysql.com/doc/>
6. Fowler, M. (2002). Patterns of Enterprise Application Architecture. Addison-Wesley.
7. Newman, S. (2021). Building Microservices (2nd ed.). O'Reilly Media.
8. Richardson, C. (2018). Microservices Patterns. Manning Publications.